

Configuración de clientes y agentes

Qué se puede configurar hoy, dónde vive cada cosa, qué no funciona y cómo lo ordenamos para que Qora sea un producto de producción.

61

configuraciones relevadas

6

lugares distintos donde viven

15

con un problema real

En una página

Lo que encontramos

- **La configuración está repartida en 6 lugares:** base de datos, archivos del repo, dashboard de ElevenLabs, variables de Railway, código fijo y WorkOS.
- **Qora no es la fuente de verdad.** Varias cosas que se editan en el panel no llegan a la llamada real, y varias que sí importan solo se tocan a mano en ElevenLabs.
- **El sistema asume un agente por cliente** aunque la base permite varios.
- **Hay restos del prototipo de un solo cliente:** productos, nombres y valores de Quintana dentro de la plataforma.

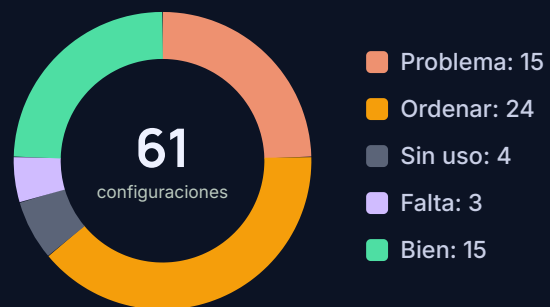
Lo que proponemos

- **Toda la configuración del cliente y del agente en la base,** versionada.
- **Herencia con override:** estándar Qora → cliente → agente. Voz, prompt y objetivo son obligatorios por agente.
- **ElevenLabs como reflejo:** Qora aplica la config por API y detecta si alguien la tocó a mano.
- **Skills en paquetes y perfiles de análisis** por rubro.
- **No es una reescritura:** son 7 fases chicas, empezando por arreglos de 1 o 2 días.

Dónde vive cada configuración



En qué estado está



Seis fuentes para una sola llamada

Para que un agente haga una llamada, Qora junta configuración de seis lugares que no se hablan entre sí. Nadie tiene la foto completa, y un cambio en uno no se refleja en los otros.



Se pierde al deployar

Lo que se edita en el panel y vive en archivos del repo (CRM) vuelve a la versión de git en cada deploy.

Solo a mano

Primer mensaje, turnos, colgar, webhooks y voz real se cambian en el dashboard de ElevenLabs, sin rastro en Qora.

Sin pantalla

Agenda, zona horaria, idioma del análisis y reglas de próxima acción no tienen UI; dos ni siquiera API.

El viaje de la velocidad de la voz

1

Panel de Qora

El superadmin pone la velocidad en 1.0. El panel dice «se aplica en la próxima llamada».

2

Base de datos

Se guarda `tts_speed = 1.0` y el agente queda «sincronizado».

3

Sincronización

Solo envía muletilla, buzón y duración. La voz no viaja.

4

Llamada real

ElevenLabs usa su propio valor: 1.2. Similitud 1.0 en vez de 0.7.

Lo mismo pasa con el modelo de voz y con la voz elegida. Solo la demo web pisa estos valores por conexión; las llamadas telefónicas nunca.

El mismo patrón, en otros lugares

Dos agentes por cliente

La llamada entra por `/voice/quintana-seguros/custom-llm`: la URL identifica al cliente, no al agente. Qora carga siempre el agente **por defecto**. Un segundo agente (por ejemplo, de cobranzas) hablaría con el prompt del primero.

La memoria del lead

El análisis extrae hechos del lead (vehículo, necesidades, señales de compra) y los guarda. **Ningún código los vuelve a poner en la conversación**. El agente recuerda resúmenes, no el perfil.

Inventario completo

Identidad y objetivo

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Nombre del cliente	Base de datos	Superadmin (UI)	—	Cliente	Bien
Cliente activo / inactivo	Base de datos	Superadmin (UI)	—	Cliente	Bien
Slug del agente	Base de datos	Superadmin, solo al crear	—	Agente	Bien
Nombre del agente	Base de datos	Superadmin (UI)	—	Agente	Bien
Agente por defecto / activo	Base de datos	Superadmin (UI)	—	Agente	Ordenar
Copia vieja de config del agente en el cliente	Base de datos	Superadmin (UI)	—	Eliminar	Sin uso
Objetivo del agente	No existe	—	—	Agente (obligatorio)	Falta

Conversación y prompt

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
System prompt (archivo)	Archivo del repo	Git (deploy)	Indirecto: texto del LLM	Agente, versionado	Ordenar
System prompt (base de datos)	Base de datos	Superadmin (UI), ignorado	—	Unificar con el anterior	Problema
Prompt de cliente y plantilla Jaumpablo	Código fijo	Git (deploy)	—	Plantillas de Qora	Ordenar
Knowledge base (DB y knowledge.md)	Base de datos	Superadmin (UI), sin efecto	—	Skill «info de la empresa»	Sin uso
Primer mensaje	Dashboard ElevenLabs	Manual (vacío hoy)	Sí, a mano	Agente	Ordenar
Idioma de la conversación	Dashboard ElevenLabs	Manual	Sí, a mano	Agente	Ordenar
Variables dinámicas por llamada	Código fijo	3 implementaciones	Sí, por llamada	Contrato único de Qora	Ordenar

Modelo de lenguaje

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Modelo del agente	Base de datos	Superadmin, la UI solo al crear	—	Estándar Qora + override	Ordenar
Temperatura y largo de respuesta	Base de datos	Superadmin (UI)	—	Estándar Qora + override	Bien
Modelo del análisis post-llamada	Variable de entorno	Railway	—	Estándar Qora	Ordenar
Frases de espera de las tools	Código fijo	—	—	Estándar Qora + idioma	Ordenar

Inventario completo

Voz

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Voz elegida	Dashboard ElevenLabs	Dashboard a mano; DB decorativa	Solo a mano	Agente (obligatoria)	Problema
Modelo de voz (TTS)	Base de datos	API, sin UI	No se sincroniza	Agente	Problema
Velocidad, estabilidad, similitud	Base de datos	Superadmin (UI)	Solo en la demo web	Agente	Problema
Sonido de fondo	Dashboard ElevenLabs	Manual	Sí, a mano	Agente	Ordenar
Voz y agente por defecto de plataforma	Variable de entorno	Railway	—	Eliminar	Sin uso

Turnos y control de la llamada

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Agresividad de turnos y timeout de turno	Dashboard ElevenLabs	Manual («eager» hoy)	Sí, a mano	Estándar Qora + override	Problema
Muletilla de espera (soft timeout)	Base de datos	API, sin UI	Sí, por API	Agente	Ordenar
Duración máxima de la llamada	Base de datos	API, sin UI	Sí, por API	Agente	Bien
Detección de buzón de voz	Base de datos	API, sin UI	Sí, por API	Agente	Ordenar
Colgar al terminar (end_call)	Dashboard ElevenLabs	Manual (apagado)	Sí, a mano	Agente	Problema
Palabras clave de reconocimiento	Dashboard ElevenLabs	Manual	Sí, a mano	Agente	Ordenar

Tools y skills

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Tools habilitadas	Base de datos	UI desactualizada	—	Agente	Problema
Campos de captura de datos	Archivo del repo	Superadmin (UI) → crm.yaml	—	Cliente / agente	Problema
Índice de skills (registry.yaml)	Archivo del repo	Git (deploy)	—	Paquetes de skills	Ordenar
Contenido de cada skill	Archivo del repo	Git (deploy)	—	Paquetes, versionado	Ordenar

Inventario completo

Memoria y análisis

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Llamadas previas en memoria (3)	Código fijo	—	—	Estándar Qora + override	Bien
Hechos del lead extraídos	Base de datos	Se guardan, nunca se usan	—	Agente (activar)	Problema
Zona horaria de la memoria	Código fijo	Fija: Buenos Aires	—	Cliente	Ordenar
Catálogo de productos y necesidades	Código fijo	Fijo: productos de Quintana	—	Perfil de análisis	Problema
Reglas de próxima acción	Base de datos	Nadie (sin API)	—	Cliente	Problema
Idioma del análisis	Base de datos	Nadie (sin API)	—	Cliente	Problema
Configuración de extracción	Base de datos	Nadie (sin uso)	—	Eliminar	Sin uso
Tipo de memoria del agente	No existe	—	—	Agente (futuro)	Falta

Agenda y reintentos (cliente)

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Agenda habilitada	Base de datos	API, sin UI	—	Cliente	Ordenar
Intentos, espera y backoff	Base de datos	API, sin UI	—	Cliente	Ordenar
Horario permitido	Base de datos	API, sin UI	—	Cliente	Ordenar
Resultados que reintentan	Base de datos	API, sin validar	—	Cliente	Ordenar
Zona horaria del cliente	Base de datos	API, sin UI	—	Cliente	Ordenar
Reintento técnico y timeouts	Código fijo	—	—	Estándar Qora	Bien
Llamadas simultáneas del discador	Variable de entorno	Railway (1 global)	—	Plan del cliente	Ordenar

Inventario completo

Plan y acceso

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
Plan	Base de datos	Superadmin (UI)	—	Cliente	Bien
Features y límites del plan	Base de datos	Superadmin (UI)	—	Cliente	Bien
Catálogo de planes	Código fijo	Git (deploy)	—	Estándar Qora	Bien
Organización de login	WorkOS	Superadmin (UI)	—	Cliente	Bien
Invitaciones de usuarios	WorkOS	Superadmin (UI)	—	Cliente	Bien
Superadmins	Variable de entorno	Railway	—	Qora	Bien
Límites de costo y minutos por agente	No existe	—	—	Agente	Falta

Integraciones y conexión

CONFIGURACIÓN	DÓNDE VIVE HOY	QUIÉN LA CAMBIA HOY	¿LLEGA A ELEVENLABS?	NIVEL PROPUESTO	ESTADO
CRM: base, tabla, mapeos, estados	Archivo del repo	Superadmin (UI), se pierde al deployar	—	Cliente (DB)	Problema
API key del CRM	Variable de entorno	Railway, una por cliente	—	Cliente (DB cifrada)	Problema
Número de teléfono saliente	Base de datos	API (ignorado al crear)	Sí, por llamada	Agente	Ordenar
Agente de ElevenLabs vinculado	Base de datos	Superadmin (UI)	—	Automático	Ordenar
URL del LLM, secreto y webhooks	Dashboard ElevenLabs	Copiar y pegar a mano	Sí, a mano	Automático	Problema
Trunk SIP y números	Dashboard ElevenLabs	Manual	Sí, a mano	Plataforma	Bien

Críticos: afectan llamadas reales hoy

La voz configurada en Qora no llega a las llamadas

Velocidad, estabilidad, similitud y modelo de voz solo se aplican en la demo web. En producción Qora dice 1.0 y 0.7; ElevenLabs usa 1.2 y 1.0, y Qora igual lo marca como «sincronizado».

Las llamadas siempre usan el agente por defecto del cliente

La URL del LLM es por cliente y resuelve el agente default (`webhook.py:902` , `initiation.py:110`). Con dos agentes por cliente, el segundo nunca habla con su propio prompt.

El CRM editado desde el panel se pierde en cada deploy

`crm.yaml` vive en el repo y se reescribe en el servidor. El próximo deploy lo vuelve a la versión de git. Además, la API key puede quedar escrita en ese archivo.

La API key faltante de un cliente apaga toda la plataforma

El chequeo de credenciales al arrancar corta el servidor completo si falta la key de un solo cliente. Sumar un cliente obliga a tocar Railway y reiniciar.

El análisis «universal» tiene los productos de Quintana

El catálogo de interés son los 9 seguros de Quintana (`catalog.py:20`). Cualquier cliente de otro rubro recibe un análisis de interés equivocado.

Lo que el análisis aprende del lead no llega al agente

Los hechos del perfil se guardan pero nadie los inyecta en la conversación (`memory.py:147` , `_format_accumulated_profile` sin uso). La memoria real es: últimos 3 resúmenes, notas y campos del lead.

Altos y medios

Altos

Panel de tools desactualizado

3 de 4 casillas son tools que ya no existen y 5 tools reales no aparecen.

Prompt de la base ignorado sin aviso

Si existe `system-prompt.md`, editar el prompt en el panel no cambia nada. La knowledge base tampoco se usa.

Configuración sin acceso

Idioma del análisis y reglas de próxima acción no se pueden cambiar ni por API. La agenda y la zona horaria no tienen pantalla.

Agente de ElevenLabs compartido

En producción, el demo de Qora y el agente de Quintana apuntan al mismo agente de ElevenLabs.

Controles solo en el dashboard

Turnos en «eager» (interrumpe), colgar apagado, primer mensaje vacío: nada de eso se ve ni se cambia desde Qora.

Detalles de creación

El teléfono se ignora al crear un agente y el default de tools tiene nombres viejos.

Medios

Valores por defecto repetidos

Los mismos números de voz están en 3 lugares; variables de plataforma guardan nombres de Quintana.

Variables por llamada en 3 lugares

Inicio, salida y demo arman los datos del lead por separado.

Documentación desactualizada

La skill y la guía de ElevenLabs describen una sincronización vieja.

Código y settings sin uso

Pantallas duplicadas, `filler_timeout_ms`, `extraction_config`, la métrica `has_facts` (siempre falsa).

Integraciones

Un usuario cliente puede disparar el test de Airtable; las escrituras del archivo no son atómicas.

Constantes fijas

Zona horaria de la memoria, reintentos técnicos y frases de espera en español.

Una configuración, tres niveles

El valor que usa un agente sale de él mismo; si no lo define, del cliente; si tampoco, del estándar de Qora. El estándar es una guía que se puede pisar en cualquier nivel.



Dónde se guarda cada cosa

La regla: el código guarda lo de Qora, la base guarda lo de cada cliente. Dar de alta un cliente pasa a ser crear filas, no carpetas ni deploys. SQLite sigue siendo la base.



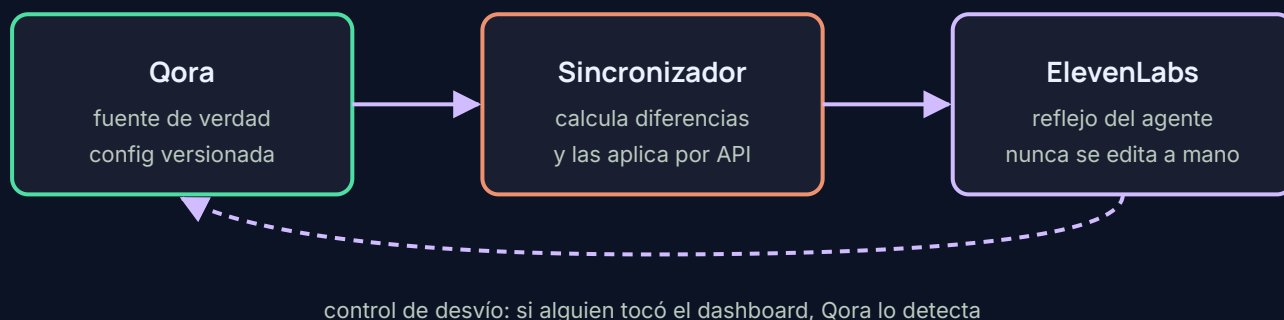
Se queda en el código

Estándares de Qora, plantillas de prompt, paquete de skills Qora, catálogo de planes, reglas técnicas de reintento. Versionado en git y revisado en PR.

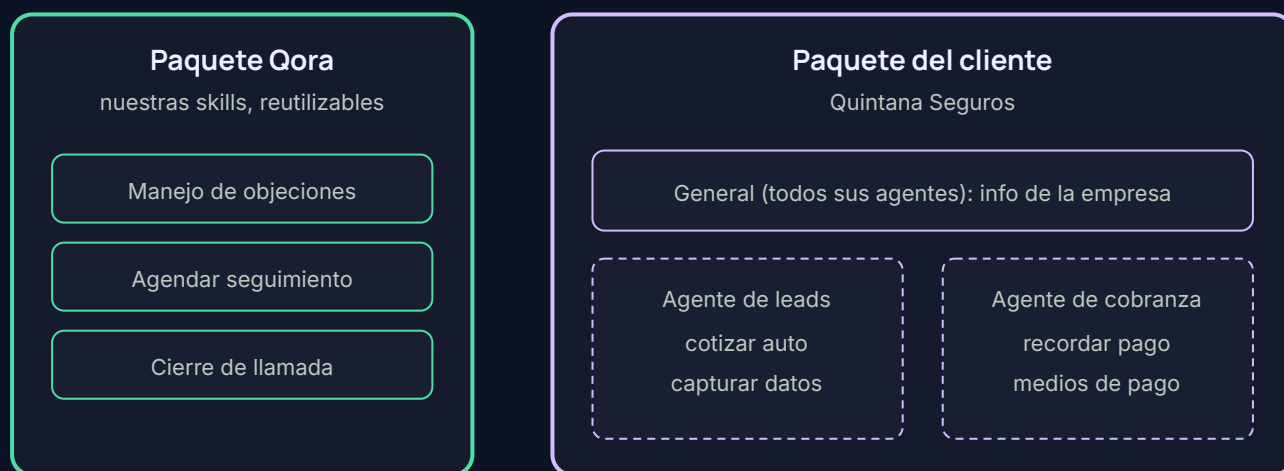
Pasa a la base

Todo lo del cliente y el agente: prompt, voz, turnos, tools, skills propias, CRM, secretos cifrados, perfil de análisis. Los archivos actuales se importan una vez.

ElevenLabs como reflejo y skills en paquetes



Un agente de ElevenLabs por cada agente de Qora, creado y actualizado por la API. Es solo configuración: tener muchos no consume capacidad. La URL del LLM pasa a identificar al agente, así cada uno usa su propio prompt.



Cada agente ve el paquete Qora, la sección general de su cliente y su propia sección. Una skill puede vivir en una sección y estar habilitada en varios agentes.

Quién toca qué

Propuesta para validar. Hoy todo es superadmin y el cliente no tiene ninguna pantalla de configuración.

CONFIGURACIÓN	QORA (SUPERADMIN)	CLIENTE
Objetivo, prompt y skills del agente	Edita	Ve y propone
Voz y parámetros de voz	Edita	Elige de un catálogo
Primer mensaje e idioma	Edita	Edita
Modelo de lenguaje y temperatura	Edita	No ve
Turnos, interrupciones y muletillas	Edita	No ve
Horarios, reintentos y zona horaria	Edita	Edita
Integración con su CRM	Edita	Conecta y mapea
Perfil de análisis	Edita	Ve
Plan, features y límites	Edita	Ve
Usuarios de su empresa	Edita	Invita

Hoy

Todas las pantallas de configuración son de superadmin. El usuario cliente solo ve dashboards y el monitor en vivo.

Por qué separar

Lo que define la calidad de la llamada (modelo, turnos, análisis) es parte del producto Qora. Lo que define el negocio (horarios, mensajes, CRM) es del cliente.

Siempre con permiso

Cada edición queda registrada como revisión con autor. Un cambio del cliente se puede auditar y revertir.

De acá a un producto ordenado

0

Arreglos rápidos

Sin cambiar el modelo. Que lo que se configura en Qora llegue de verdad a las llamadas.

1

Configuración del agente en la base

Prompt, voz, turnos y tools en revisiones versionadas. Los archivos pasan a ser importación inicial.

2

ElevenLabs como reflejo

Sincronizador completo, control de desvío y ruteo por agente (no por cliente).

3

Integraciones y secretos

CRM y API keys por cliente en la base, cifradas. Un cliente mal configurado no tumba a los demás.

4

Paquetes de skills

Paquete Qora + paquete del cliente con secciones por agente.

5

Perfiles de análisis

Catálogo de productos y reglas por vertical o cliente.

6

Arnés de onboarding y MCP

Dar de alta un cliente es una operación guiada; el MCP opera sobre esta misma API.

Fase 0 en detalle

- Enviar voz y parámetros de voz a ElevenLabs en la sincronización, y que «sincronizado» compare valores reales.
- Pasar turnos a una agresividad normal y habilitar «colgar» en el agente de producción.
- Corregir la lista de tools del panel y el default de la columna.
- Aceptar el teléfono al crear un agente.
- Separar el agente de ElevenLabs del demo y el de Quintana.
- Activar los hechos del perfil del lead en la memoria del agente.
- Exponer por API idioma del análisis y reglas de próxima acción.

Lo que necesito de vos

Con estas cuatro respuestas arranco la fase 0 y escribo la propuesta formal de la fase 1.

¿Qué puede tocar el cliente?

Propuesta: el cliente ve todo, pero edita solo lo de negocio (horarios, primer mensaje, datos de su empresa, voz desde un catálogo curado). Modelo, turnos y análisis quedan en manos de Qora.

¿Un agente de ElevenLabs por cada agente de Qora?

Propuesta: sí, creado y mantenido por la API, nunca a mano. Un agente de ElevenLabs es solo configuración, no cuesta capacidad. Si algún día conviene compartir, ElevenLabs permite pisar voz, prompt y primer mensaje por llamada.

¿Perfiles de análisis por rubro o por cliente?

Propuesta: por rubro (seguros, cobranzas, inmobiliaria) con override por cliente. Arranca con «seguros» = lo de hoy.

¿Por dónde empezamos?

Propuesta: fase 0 primero (arreglos rápidos, 1 o 2 días) y después fase 1. Las fases 2 a 6 se diseñan con OpenSpec antes de tocar código.